

R.M.K ENGINEERING COLLEGE

Department of Computer Science and Design

SALESFORCE CRM PROJECT DOCUMENTATION

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Submitted as part of the
Salesforce Certified Platform Developer Certification Program (TRINITY)

Submitted By Vemuru Divya Charishma

Salesforce Certified Platform Developer

Smart Bridge

Document Version 2.0 (Revised & Consolidated)

September 2026

1. Project Overview

WhatNext Vision Motors is a Salesforce-based CRM project created to manage the main activities of an automotive dealership in one place. Instead of keeping vehicle, customer, dealer, order, test-drive, and service information separately, the project brings these records together inside a single Lightning application.

The project was designed around a simple dealership workflow. Staff can add vehicles and maintain their stock details, register customers, manage dealers, create vehicle orders, schedule test drives, and record service requests. Relationships between these records make it easier to move from one activity to another without repeatedly entering the same information. Automation was added for the parts of the process that would otherwise require repeated manual work. For example, the system checks vehicle stock before an order is confirmed, assigns a dealer through Flow when an order is created, and sends a reminder before a scheduled test drive. Apex is used where programmatic logic is needed, while Salesforce configuration features handle the remaining business requirements.

The application is presented through Salesforce Lightning pages, with separate tabs and organized record layouts. Reports and dashboards are also used to give users a quick view of stock, orders, and test-drive activity.

Project Objectives

The project was developed with the following objectives:

2.1 Centralize Automotive Information

Bring vehicle, customer, dealer, order, test-drive, and service details together in a single Salesforce system for easier management and access.

2.2 Improve Order Processing

Make the vehicle ordering process more organized by checking stock availability before an order is confirmed.

2.3 Reduce Manual Effort

Use Salesforce Flow and Apex to automate regular tasks and reduce the amount of manual work required from users.

2.4 Maintain Data Accuracy

Use validation rules and Apex-based checks to prevent incorrect entries and avoid errors during transactions.

2.5 Simplify User Interaction

Create an easy-to-use Lightning application with clear navigation, well-organized pages, and the required information available when users need it.

3. Phase 1 — Requirement Analysis and Planning

The first phase was about understanding how an automotive dealership would use the CRM and then converting those needs into Salesforce features. The work began by identifying the information that needs to be stored, the way different records are connected, the activities that should be automated, and the level of access different users may require.

3.1 Business Requirements

The project requirements were based on the normal flow of dealership operations. The system needed to handle both sales-related activities and customer support after a purchase. The main requirements were:

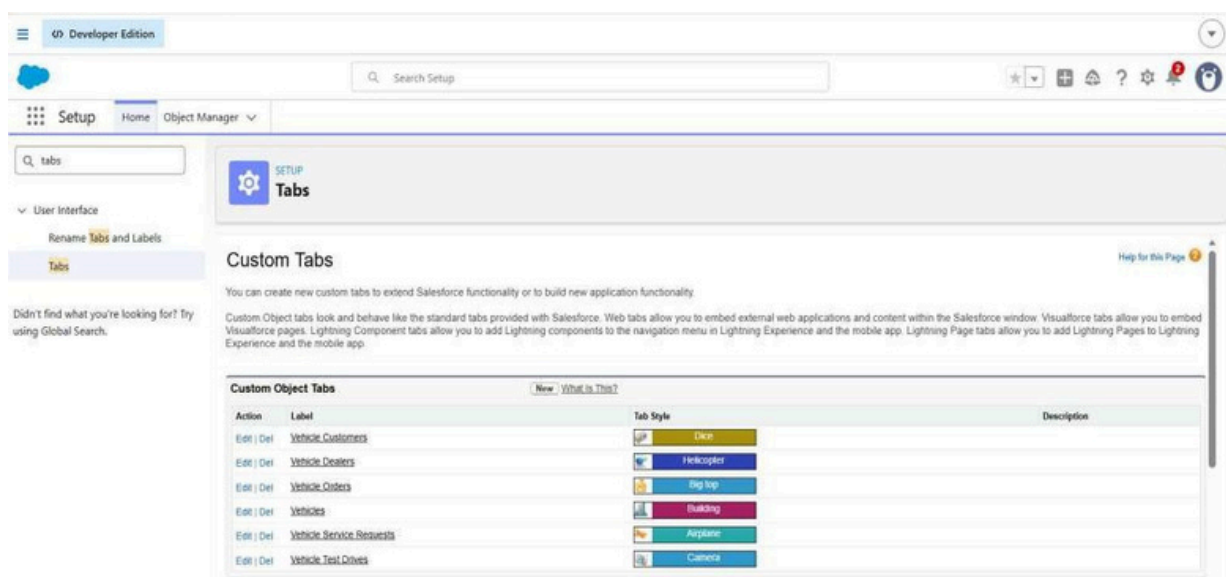
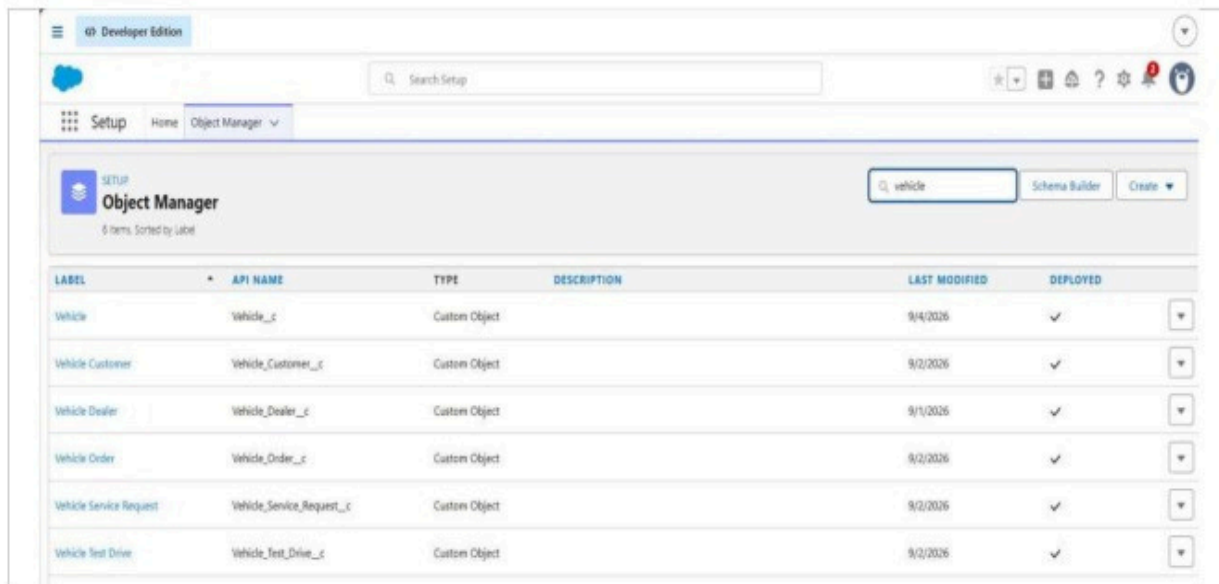
- Store vehicle information such as model, price, stock, and availability.
- Maintain customer details for purchases, test drives, and service activities.
- Store dealer information and contact details.
- Create and track vehicle orders.
- Connect orders with the relevant customer, vehicle, and dealer.
- Schedule and manage test drives.
- Record service requests raised for customers and vehicles.

3.2 Project Scope

- Vehicle Management: Stores vehicle details, pricing, stock quantity, availability, and dealer information.
- Dealer Management: Maintains dealership details and contact information.
- Customer Management: Stores customer records used across sales, test-drive, and service activities.
- Order and Test Drive Management: Handles vehicle orders and scheduled test drives by connecting customers with their selected vehicles.
- Service Request Management: Records customer service requirements and links them to the appropriate customer and vehicle records.

3.3 Data Model

The data model was planned around the main records used in the dealership process. Vehicles are connected with orders and test drives, while customers can be connected with orders, test drives, and service requests. Dealer information is also connected to vehicle transactions. These relationships allow users to open related information from a record instead of maintaining the same information in multiple places.



3.4 Security Model

Security was considered while designing the application.

- Salesforce Profiles
- Roles and Role Hierarchy
- Permission Sets
- Object- and Field-level permissions
- Sharing Rules were used as the main access-control mechanisms.

4. Phase 2 — Salesforce Development: Backend & Configuration

In the second phase, the planned CRM structure was implemented in Salesforce. This included creating the custom objects and fields, connecting related records, adding validation rules, building Flow automations, and implementing Apex logic for order processing.

4.1 Setup Environment & DevOps Workflow

The application was developed in Salesforce Lightning Experience using a Developer Edition org. Salesforce was used as the main environment for configuring the data model, creating automation, developing Apex, preparing the user interface, and setting up reports and security.

4.2 Custom Objects

Six custom objects were created to represent the main areas of the WhatNext Vision Motors system:

Vehicle: Stores vehicle name/model, price, stock quantity, availability status, and dealer details.

I Vehicle Dealer: Stores dealer name, location, city, and contact information.

I Vehicle Customer: Stores customer information needed for purchases, test drives, and service.

I Vehicle Order: Records a vehicle purchase and connects the customer, vehicle, dealer, and order status.

I Vehicle Test Drive: Stores test-drive bookings, including customer, vehicle, appointment date, and status.

I Vehicle Service Request: Records customer service requirements and connects them with the related vehicle and customer.

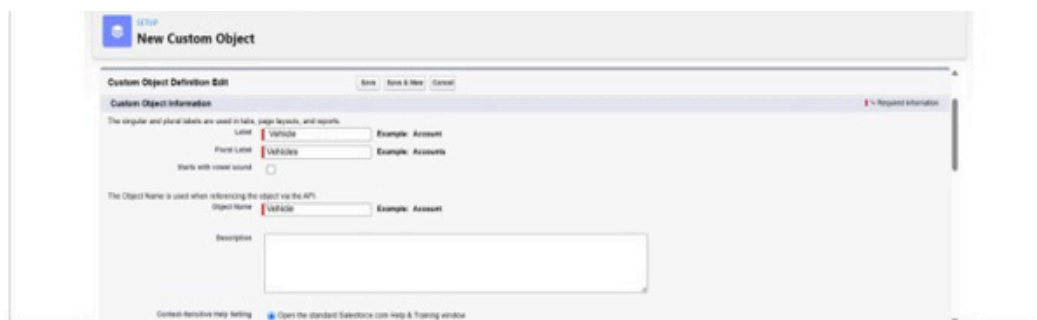


Figure 4.1 — Creating the Vehicle custom object: label, plural label, and API name.

The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Setup' menu is open, and 'Object Manager' is selected. The 'New Custom Object' page is displayed. A yellow banner at the top states: 'Permissions for this object are disabled for all profiles by default. You can enable object permissions in permission sets or by editing custom profiles. [Get more info](#) [Go to object permissions page](#)'. Below this, the 'Custom Object Definition Edit' section is active, with 'Save', 'Save & New', and 'Cancel' buttons. The 'Custom Object Information' section is expanded, showing fields for 'Label' (Vehicle Dealer), 'Plural Label' (Vehicle Dealers), and 'Object Name' (Vehicle_Dealer). Examples for 'Account' and 'Accounts' are provided. A 'Description' field is also visible.

Figure 4.3 — Creating the Vehicle Dealer custom object.

The screenshot shows the Salesforce Setup interface in Developer Edition, specifically the 'New Custom Object' page for 'Vehicle Customer'. The 'Custom Object Information' section is expanded, showing fields for 'Label' (Vehicle Customer), 'Plural Label' (Vehicle Customers), and 'Object Name' (Vehicle_Customer). Examples for 'Account' and 'Accounts' are provided. A 'Description' field is also visible. The 'Starts with vowel sound' checkbox is unchecked. A red icon and 'Required Information' text are visible in the top right corner of the section.

Figure 4.4 — Creating the Vehicle Customer custom object.

Setup Home Object Manager

SETUP New Custom Object

Custom Object Information ⓘ Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

33°C Mostly sunny

Figure 4.5 — Creating the Vehicle Order custom object.

Setup Home Object Manager

SETUP New Custom Object

permission sets or by editing custom profiles. [Tell me more!](#) [Don't show this message again](#)

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information ⓘ Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Figure 4.6 — Creating the Vehicle Test Drive custom object.

Search Setup

Setup Home Object Manager

SETUP
New Custom Object

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information ⓘ Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

Figure 4.7 — Creating the Vehicle Service Request custom object.

4.2.1 Custom Tabs

A dedicated custom tab was created for every custom object so that each entity is independently navigable within the Lightning application.

Developer Edition

Search Setup

Setup Home Object Manager

SETUP
Tabs

Custom Tabs

You can create new custom tabs to extend Salesforce functionality or to build new application functionality.

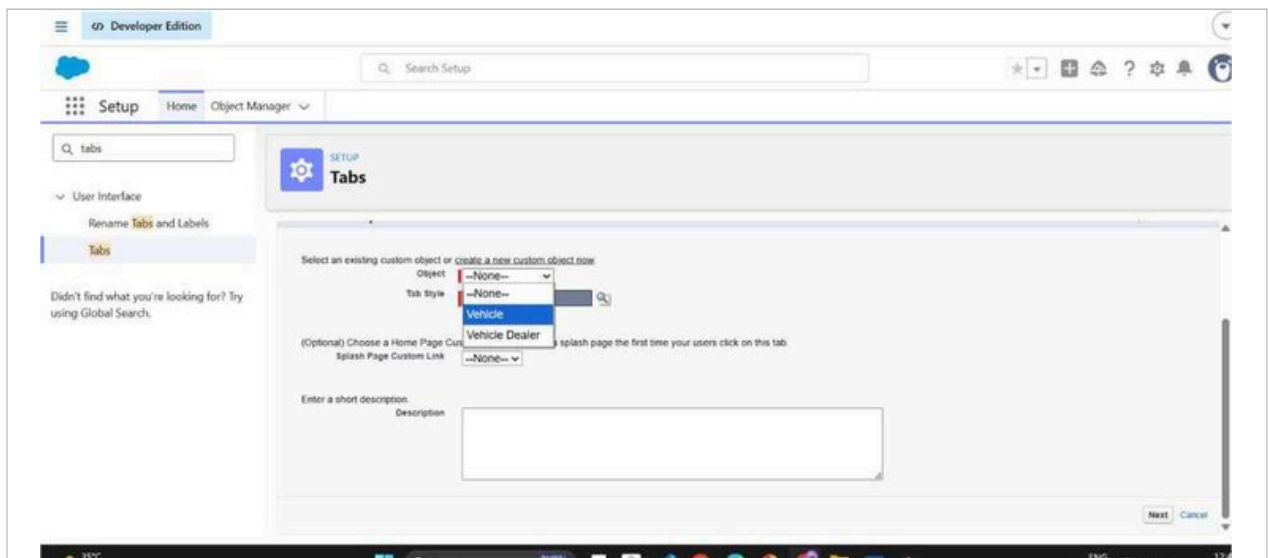
Custom Object tabs look and behave like the standard tabs provided with Salesforce. Web tabs allow you to embed external web applications and content within the Salesforce window. Visualforce tabs allow you to embed Visualforce pages. Lightning Component tabs allow you to add Lightning components to the navigation menu in Lightning Experience and the mobile app. Lightning Page tabs allow you to add Lightning Pages to Lightning Experience and the mobile app.

Custom Object Tabs	New	What Is This?
No Custom Object Tabs have been defined		

Web Tabs	New	What Is This?
No Web Tabs have been defined		

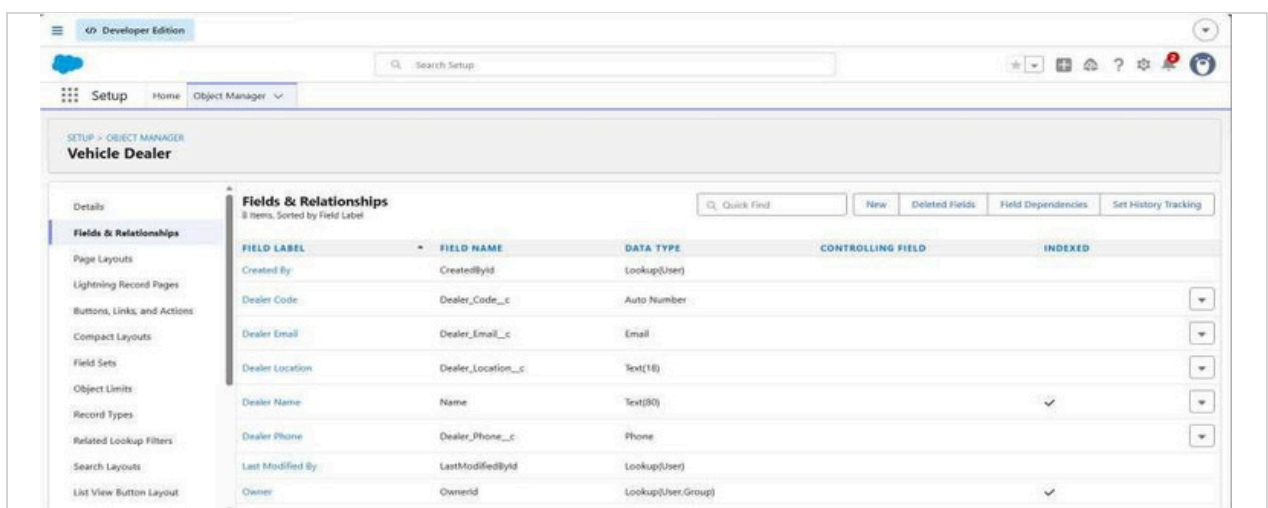
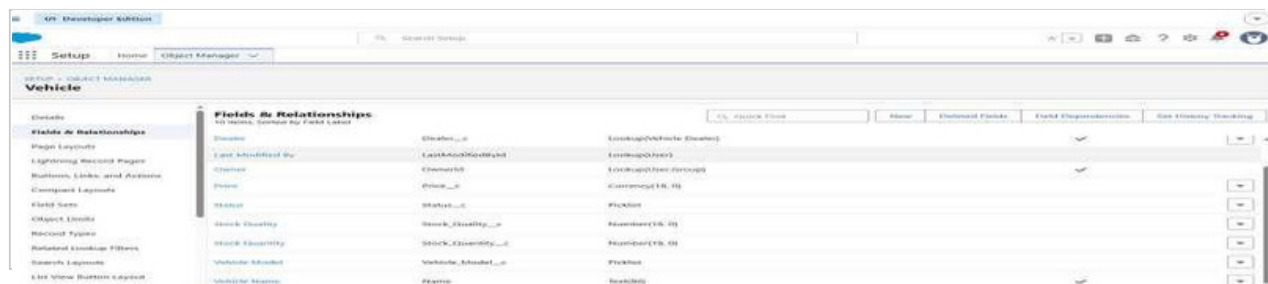
Visualforce Tabs	New	What Is This?

Figure 4.8 — The Custom Tabs setup page before tab creation.



4.3 Fields

Custom fields were created on every object to capture exactly the information required by that entity, including text, number, currency, date, email, phone, picklist, and lookup fields. The final field sets are summarized below.



Developer Edition

Setup

Home

Object Manager

Vehicle Customer

Details

Fields & Relationships

8 Items, Sorted by Field Label

Quick Find

New

Deleted Fields

Field Dependencies

Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User.Group)		✓
Preferred Vehicle Type	Preferred_Vehicle_Type__c	Picklist		
Vehicle Customer Address	Vehicle_Customer_Address__c	Text(18)		
Vehicle Customer Email	Vehicle_Customer_Email__c	Email		
Vehicle Customer Phone	Vehicle_Customer_Phone__c	Phone		
Vehicle Customer Name	Name	Text(80)		✓

Developer Edition

Setup

Home

Object Manager

Vehicle Order

Details

Fields & Relationships

8 Items, Sorted by Field Label

Quick Find

New

Deleted Fields

Field Dependencies

Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Order Date	Order_Date__c	Date		
Owner	OwnerId	Lookup(User.Group)		✓
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Order Name	Name	Text(80)		✓
Vehicle Order Status	Status__c	Picklist		

Developer Edition

Setup

Home

Object Manager

Vehicle Test Drive

Details

Fields & Relationships

8 Items, Sorted by Field Label

Quick Find

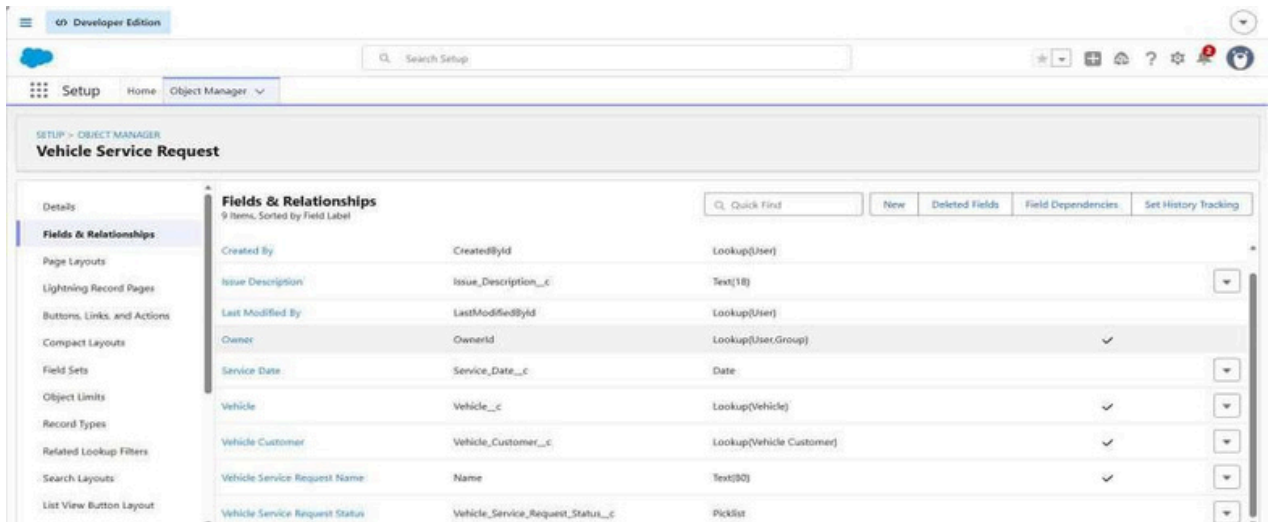
New

Deleted Fields

Field Dependencies

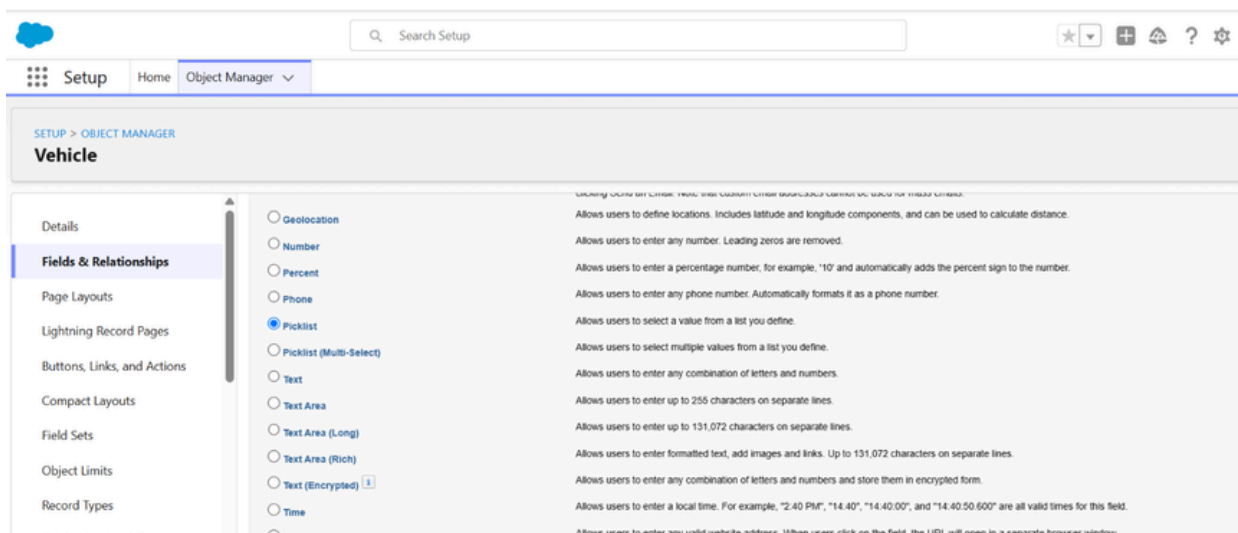
Set History Tracking

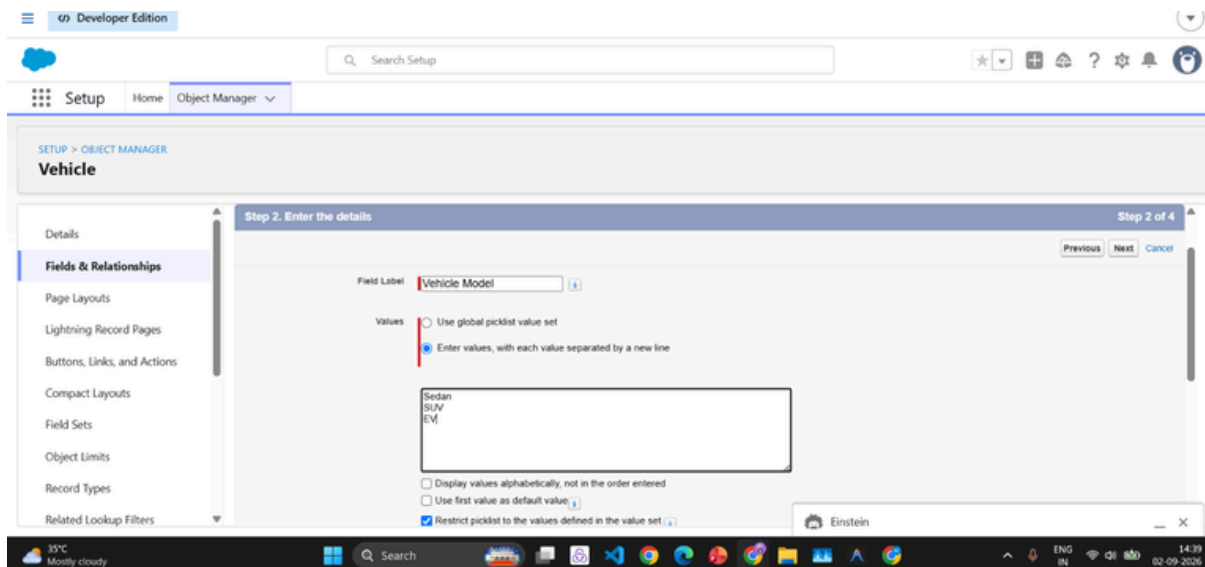
FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User.Group)		✓
Test Drive Date	Test_Drive_Date__c	Date		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Customer	Vehicle_Customer__c	Lookup(Vehicle Customer)		✓
Vehicle Test Drive Name	Name	Text(80)		✓
Vehicle Test Drive Status	Vehicle_Test_Drive_Status__c	Picklist		



4.3.1 Representative Field-Creation

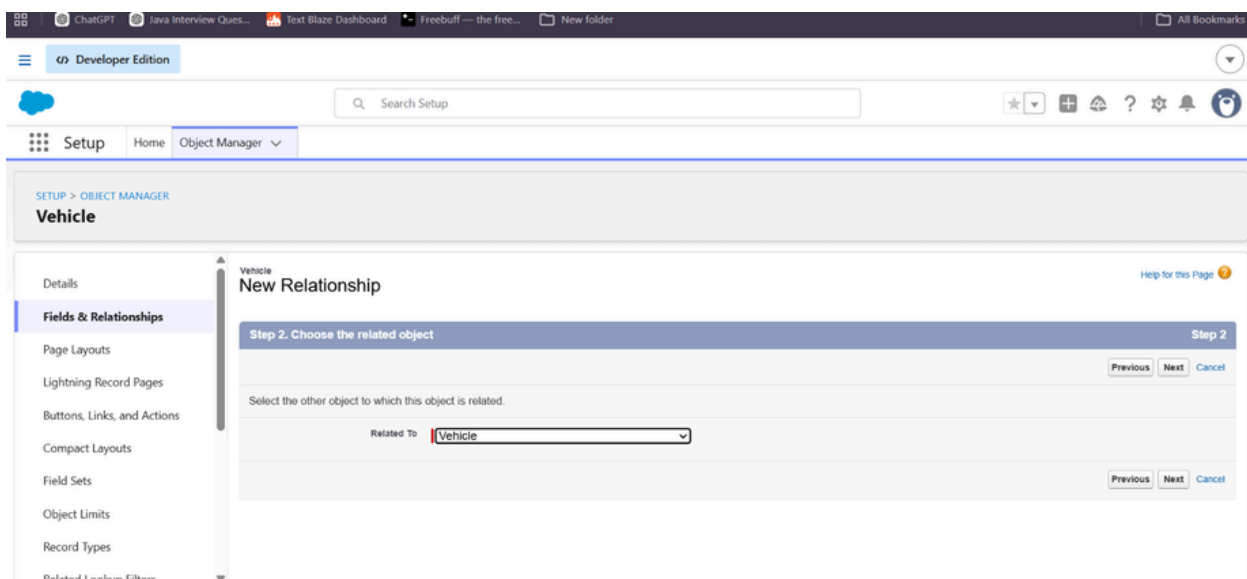
The field-creation process was carried out based on the type of information required for each Salesforce object. For the Vehicle object, the Vehicle Model field was created as a Picklist so that users could select the appropriate vehicle category from a predefined list. During the setup, the field type was selected, the field label and API name were provided, and the required picklist values were added. Values such as Sedan, SUV, and EV were included based on the vehicle categories used in the project. This approach makes data entry more consistent and avoids users entering different spellings or formats for the same vehicle type. The same general process was followed while creating the other fields across the Vehicle, Dealer, Customer, Order, Test Drive, and Service Request objects.

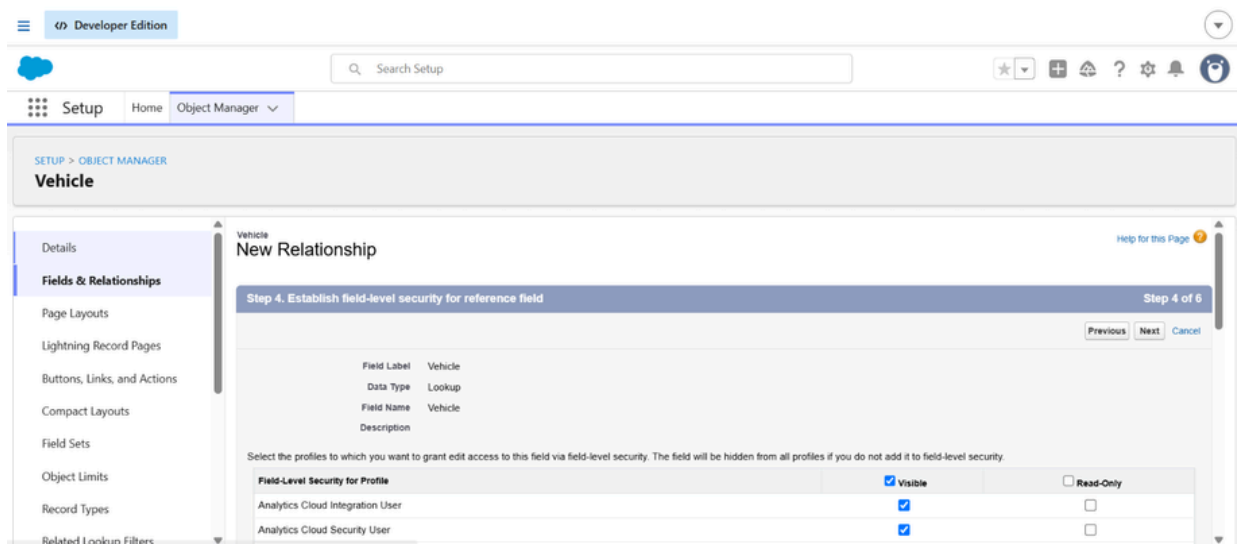




4.4 Objects and it's Relationships

Object relationships were created to connect the different records involved in the dealership process. In the project, Vehicle Orders and Vehicle Test Drives are linked with the corresponding Vehicle and Vehicle Customer records. Similarly, Vehicle Service Requests are connected with the related Vehicle and Customer records. These relationships allow users to access related information easily from a record and help maintain consistency across the application. For example, when viewing an order, the user can identify the customer who placed it and the vehicle selected for the purchase. The relationship fields were created using Salesforce Lookup relationships, and the required field-level security settings were configured during the setup process.



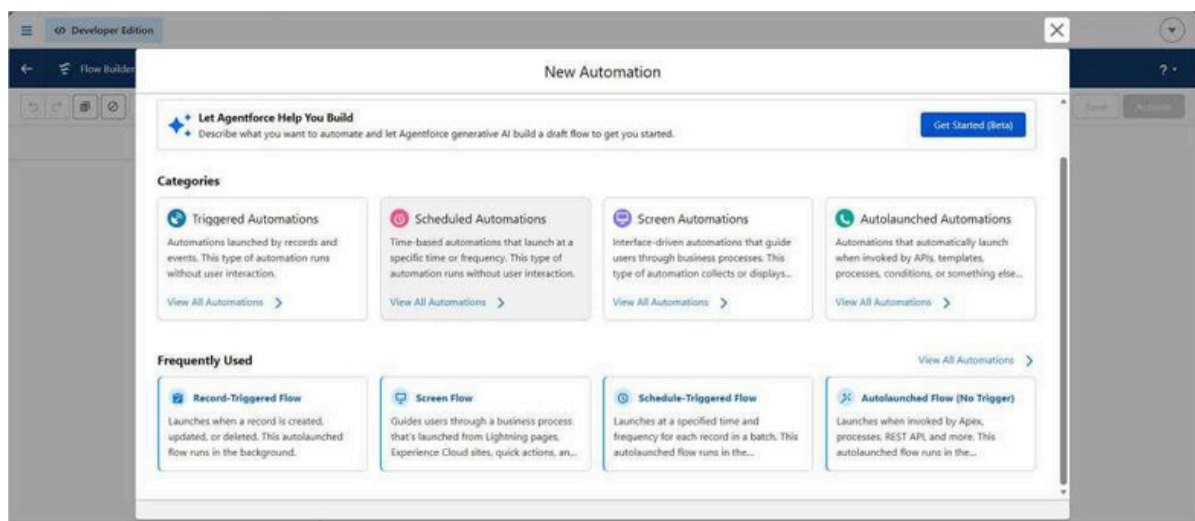


4.5 Validation Rule — Vehicle Stock

A validation rule was added to make sure that a vehicle order is confirmed only when the selected vehicle has sufficient stock. Before completing the order, Salesforce checks the available stock quantity of the associated vehicle. If the stock quantity is zero, the user is prevented from confirming the order and the configured error message is displayed. This helps avoid orders being created for vehicles that are currently unavailable and keeps the order information consistent with the vehicle inventory.

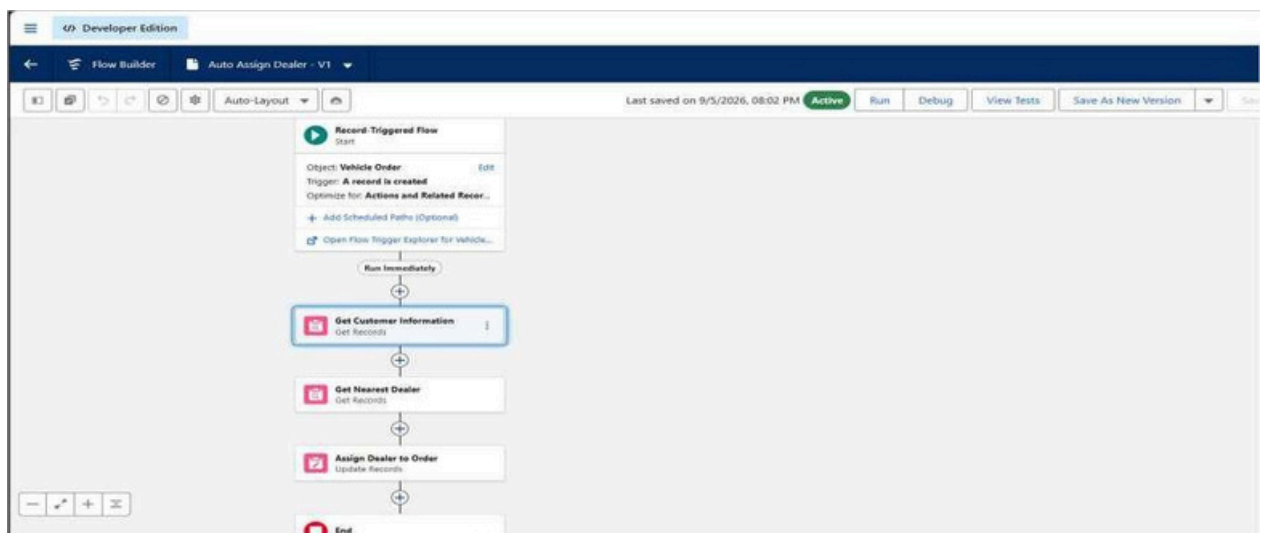
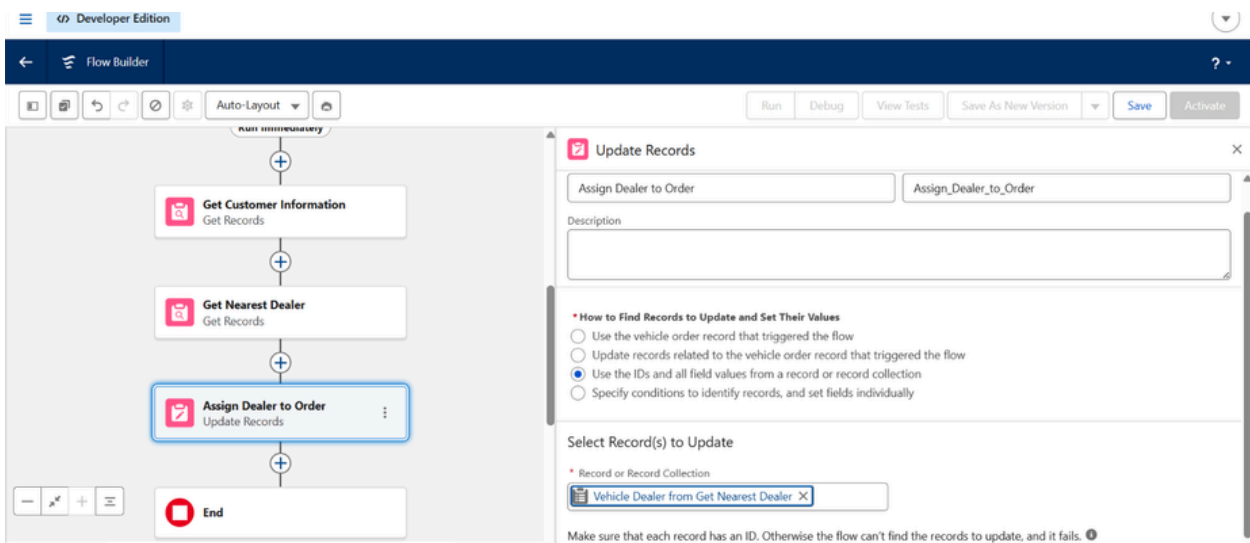
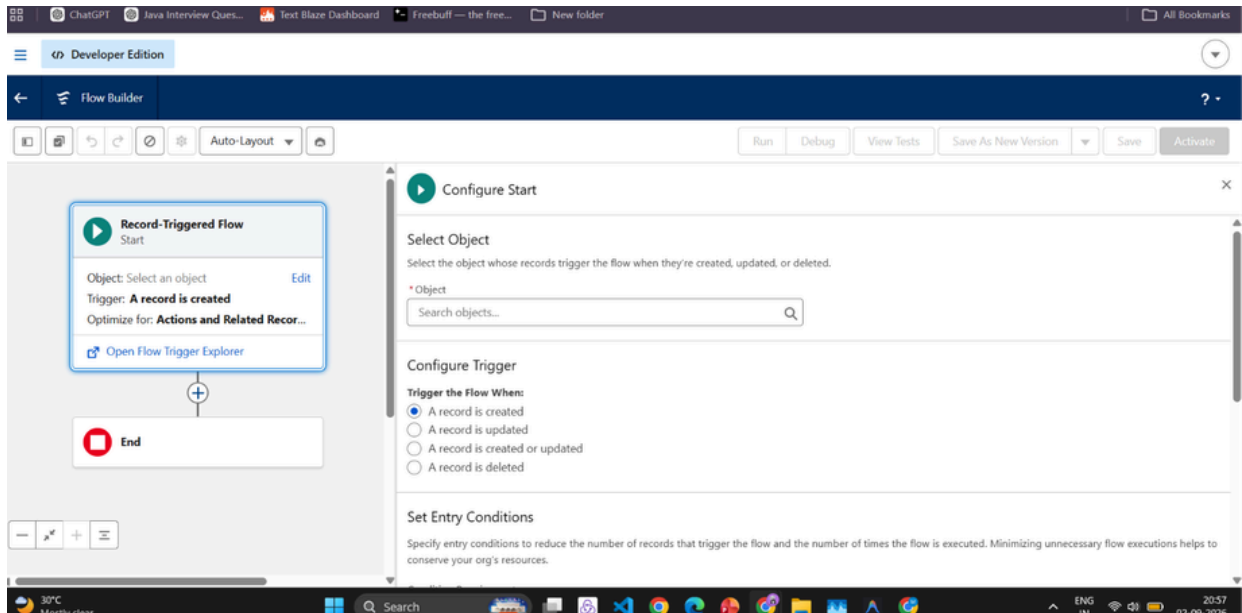
4.6 Flow

Flow was used to automate repetitive activities in the WhatNext Vision Motors application. Instead of requiring users to perform every step manually, the configured flows respond to specific record events and carry out the required actions automatically. Two record-triggered flows were developed for the project. The first flow handles dealer assignment when a new vehicle order is created, while the second flow sends a reminder email before a scheduled test drive. These automations help reduce manual work and make the dealership workflow more systematic.



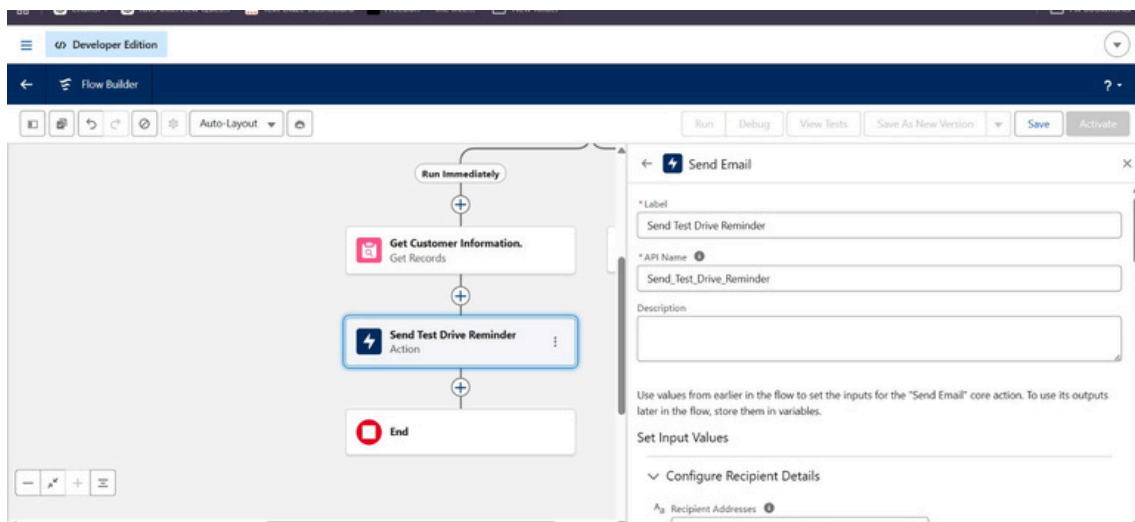
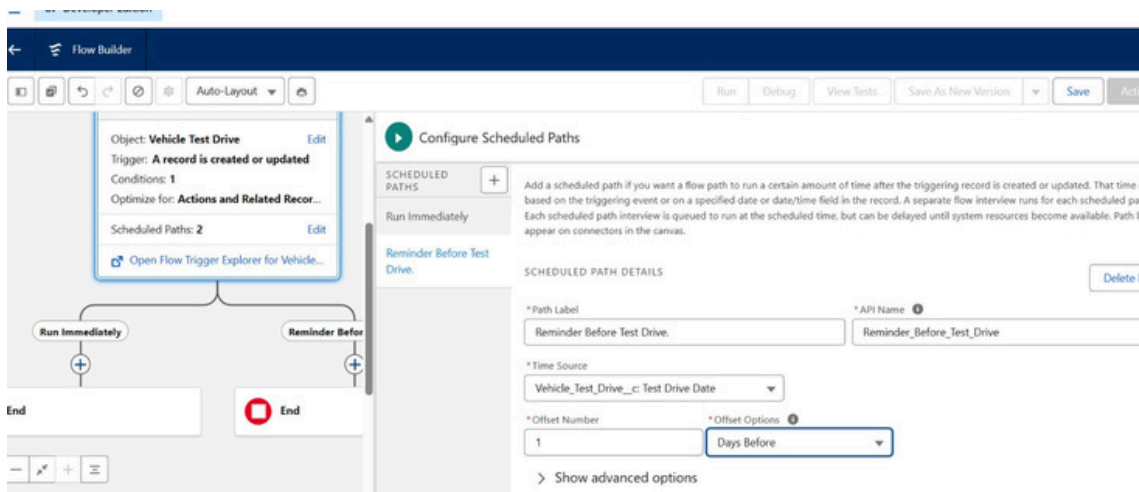
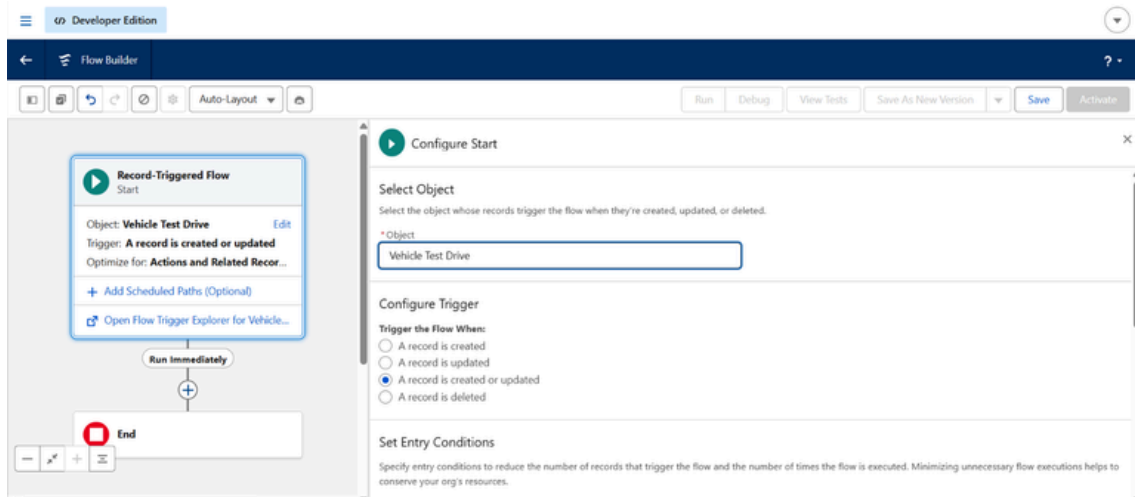
4.6.1 Auto Assign Dealer

When a new Vehicle Order is created, the flow automatically checks the customer's information, finds a suitable nearby dealer with vehicle availability, and assigns that dealer to the order. This avoids the need for users to assign dealers manually.



4.6.2 Test Drive Reminder

This flow is created for the Vehicle Test Drive object. It automatically sends a reminder email to the customer one day before the scheduled test drive, helping ensure that the customer is informed about the upcoming appointment without requiring manual reminders.



4.7 Apex Classes and Triggers

4.7.1 Apex Classes

Apex classes were used in the project to handle the business logic related to vehicle orders and stock management. The main responsibilities include:

- **Vehicle Order Processing:** Handles the processing of vehicle orders and the related business operations.
- **Stock Validation:** Checks the availability of the selected vehicle and ensures that the order follows the defined stock conditions.
- **Order Management:** Supports updating and processing vehicle order records based on their current status.
- **Reusable Business Logic:** Keeps the main business logic inside Apex classes, making the code easier to maintain and test.
- **Automation Support:** Apex classes work along with triggers and Salesforce automation to perform the required operations automatically and efficiently.



```
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(
4         List<Vehicle_Order__c> newOrders,
5         Map<Id, Vehicle_Order__c> oldOrders,
6         Boolean isBefore,
7         Boolean isAfter,
8         Boolean isInsert,
9         Boolean isUpdate
10    ) {
11
12        if (isBefore) {
13            if (isInsert || isUpdate) {
14                preventOrderIfOutOfStock(newOrders);
15            }
16        }
17
18        if (isAfter) {
19            if (isInsert || isUpdate) {
20                updateStockOnOrderPlacement(newOrders);
21            }
22        }
23    }
```



```
26
27 private static void preventOrderIfOutOfStock(
28     List<Vehicle_Order__c> orders
29 ) {
30
31     Set<Id> vehicleIds = new Set<Id>();
32
33     for (Vehicle_Order__c order : orders) {
34         if (order.Vehicle__c != null) {
35             vehicleIds.add(order.Vehicle__c);
36         }
37     }
38
39     if (!vehicleIds.isEmpty()) {
40
41         Map<Id, Vehicle__c> vehicleStockMap =
42             new Map<Id, Vehicle__c>();
43
44         for (Vehicle__c vehicle : [
45             SELECT Id, Stock_Quantity__c
46             FROM Vehicle__c
47             WHERE Id IN :vehicleIds
48         ])
```

```
    {
        vehicleStockMap.put(vehicle.Id, vehicle);
    }
}

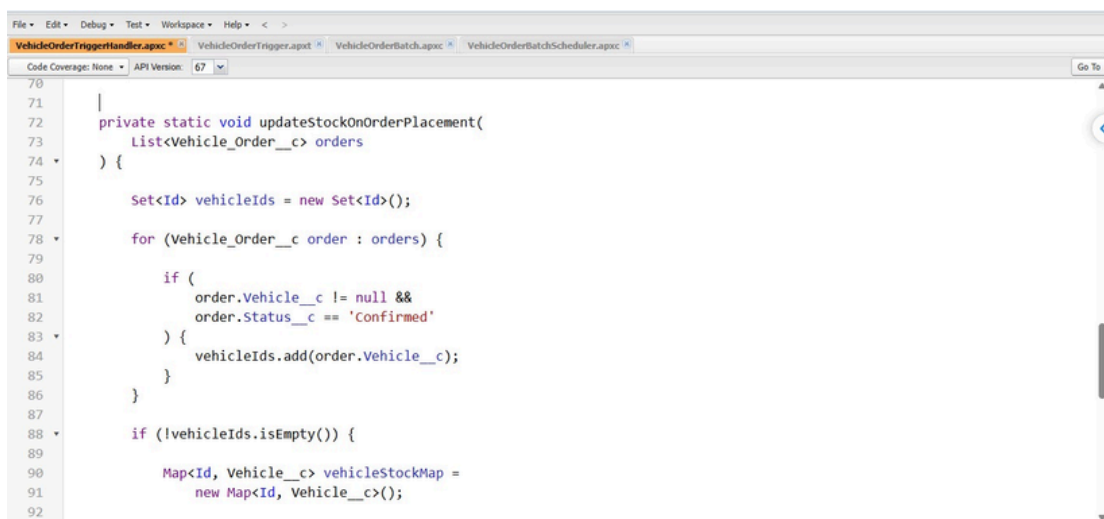
for (Vehicle_Order__c order : orders) {
    if (vehicleStockMap.containsKey(order.Vehicle__c)) {
        Vehicle__c vehicle =
            vehicleStockMap.get(order.Vehicle__c);
        if (vehicle.Stock_Quantity__c <= 0) {
            order.addError(
                'This vehicle is out of stock. Order cannot be placed.'
            );
        }
    }
}
```

4.7.2 Apex Triggers

The Vehicle Order process also uses an Apex trigger to automatically handle important actions when an order record is created or updated. The trigger performs the required checks and updates, including:

- Checking Stock Availability: Verifies whether the selected vehicle has sufficient stock.
- Preventing Invalid Orders: Stops the order from being processed when the required stock conditions are not satisfied.
- Updating Vehicle Stock: Reduces the available vehicle stock after a confirmed order.
- Order Status Processing: Handles the required actions based on changes in the order status.

The trigger follows a handler pattern, where the trigger mainly responds to the record event while the actual business logic is handled separately. This keeps the code organized and makes it easier to maintain and test.



```
70
71
72 | private static void updateStockOnOrderPlacement(
73 |     List<Vehicle_Order__c> orders
74 | ) {
75
76 |     Set<Id> vehicleIds = new Set<Id>();
77
78 |     for (Vehicle_Order__c order : orders) {
79
80 |         if (
81 |             order.Vehicle__c != null &&
82 |             order.Status__c == 'Confirmed'
83 |         ) {
84 |             vehicleIds.add(order.Vehicle__c);
85 |         }
86 |     }
87
88 |     if (!vehicleIds.isEmpty()) {
89
90 |         Map<Id, Vehicle__c> vehicleStockMap =
91 |             new Map<Id, Vehicle__c>();
92
```

```
Map<Id, Vehicle__c> vehicleStockMap =
    new Map<Id, Vehicle__c>();

for (Vehicle__c vehicle : [
    SELECT Id, Stock_Quantity__c
    FROM Vehicle__c
    WHERE Id IN :vehicleIds
]) {
    vehicleStockMap.put(vehicle.Id, vehicle);
}

List<Vehicle__c> vehiclesToUpdate =
    new List<Vehicle__c>();

for (Vehicle_Order__c order : orders) {

    if (vehicleStockMap.containsKey(order.Vehicle__c)) {

        Vehicle__c vehicle =
            vehicleStockMap.get(order.Vehicle__c);
```

```

        if (vehicle.Stock_Quantity__c > 0) {
            vehicle.Stock_Quantity__c -= 1;
            vehiclesToUpdate.add(vehicle);
        }
    }
}

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}
}
}

```

```

File • Edit • Debug • Test • Workspace • Help • < >
VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex • VehicleOrderBatch.apex • VehicleOrderBatchScheduler.apex
Code Coverage: None • API Version: 67 • Go To
1 • trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {
2
3     VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap, trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);
4
5 }

```

```

File • Edit • Debug • Test • Workspace • Help • < >
VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex • VehicleOrderBatch.apex • VehicleOrderBatchScheduler.apex
Code Coverage: None • API Version: 67 • Go To
1 • global class VehicleOrderBatch implements Database.Batchable<SObject> {
2
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4
5         return Database.getQueryLocator([
6             SELECT Id, Status__c, Vehicle__c
7             FROM Vehicle_Order__c
8             WHERE Status__c = 'Pending'
9         ]);
10    }
11
12    global void execute(
13        Database.BatchableContext bc,
14        List<Vehicle_Order__c> orderList
15    ) {
16
17        Set<Id> vehicleIds = new Set<Id>();
18
19        for (Vehicle_Order__c order : orderList) {
20
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25    }
26 }

```

```

File • Edit • Debug • Test • Workspace • Help • < >
VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex • VehicleOrderBatch.apex • VehicleOrderBatchScheduler.apex
Code Coverage: None • API Version: 67 • Go To
27
28 • if (!vehicleIds.isEmpty()) {
29
30     Map<Id, Vehicle__c> vehicleStockMap =
31         new Map<Id, Vehicle__c>();
32
33     for (Vehicle__c vehicle : [
34         SELECT Id, Stock_Quantity__c
35         FROM Vehicle__c
36         WHERE Id IN :vehicleIds
37     ]) {
38         vehicleStockMap.put(vehicle.Id, vehicle);
39     }
40
41     List<Vehicle_Order__c> ordersToUpdate =
42         new List<Vehicle_Order__c>();
43
44     for (Vehicle_Order__c order : orderList) {
45
46         if (vehicleStockMap.containsKey(order.Vehicle__c)) {
47
48             order.Stock_Quantity__c -= 1;
49             ordersToUpdate.add(order);
50         }
51     }
52 }
53
54 if (!ordersToUpdate.isEmpty()) {
55     update ordersToUpdate;
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }

```

```

File • Edit • Debug • Test • Workspace • Help • < >
VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex • VehicleOrderBatch.apex • VehicleOrderBatchScheduler.apex
Code Coverage: None • API Version: 67 • Go To
50
51     Vehicle__c vehicle =
52         vehicleStockMap.get(order.Vehicle__c);
53
54     if (vehicle.Stock_Quantity__c > 0) {
55         order.Stock_Quantity__c -= 1;
56         ordersToUpdate.add(order);
57     }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }

```

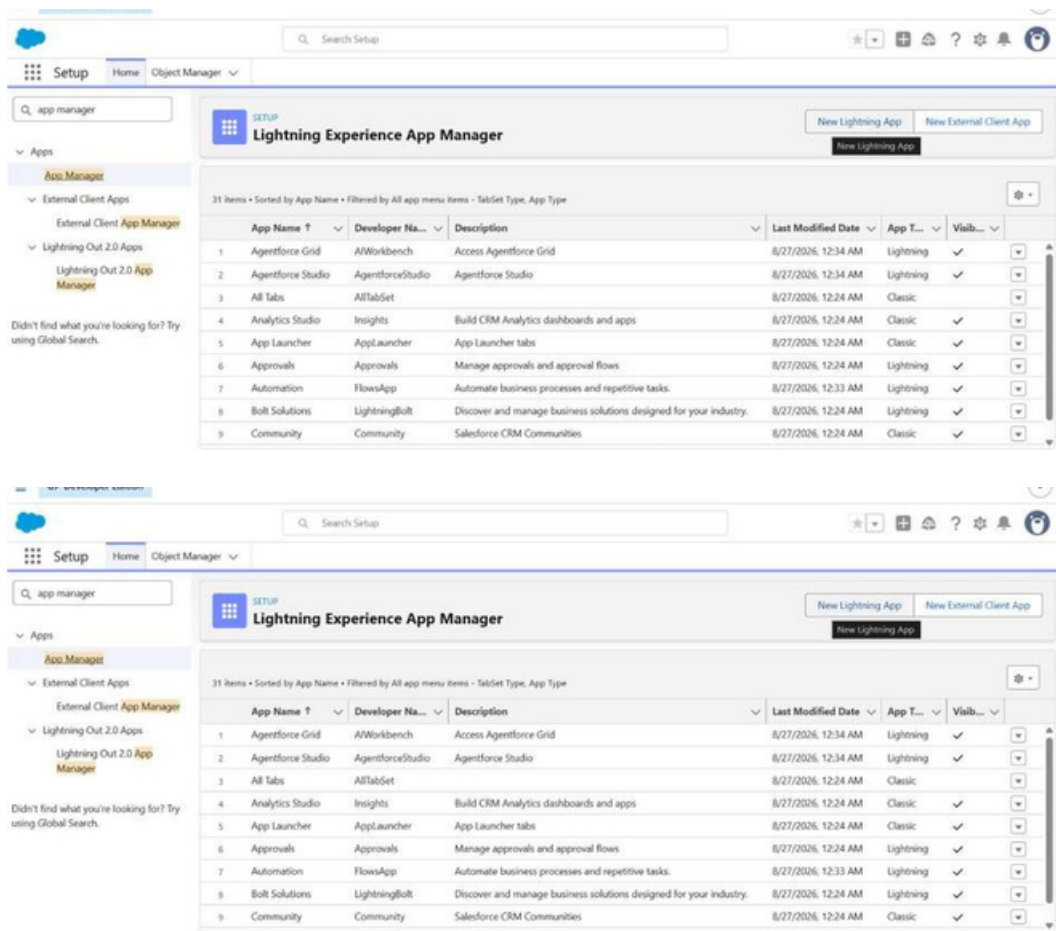
Schedule Apex							
Action	Job Name ↑	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Vehicle Order Processing	Divya Charishma Vemuru	9/8/2026, 8:26 AM		9/8/2026, 12:00 PM	Scheduled Apex	08ef00000ZcAyy

5. Phase 3 — UI/UX Development & Customization

The third phase focused on making the Salesforce application practical for everyday use. The Lightning App was organized around the main dealership records, and page layouts, dynamic forms, user access, reports, dashboards, and record pages were configured to make the information easier to work with.

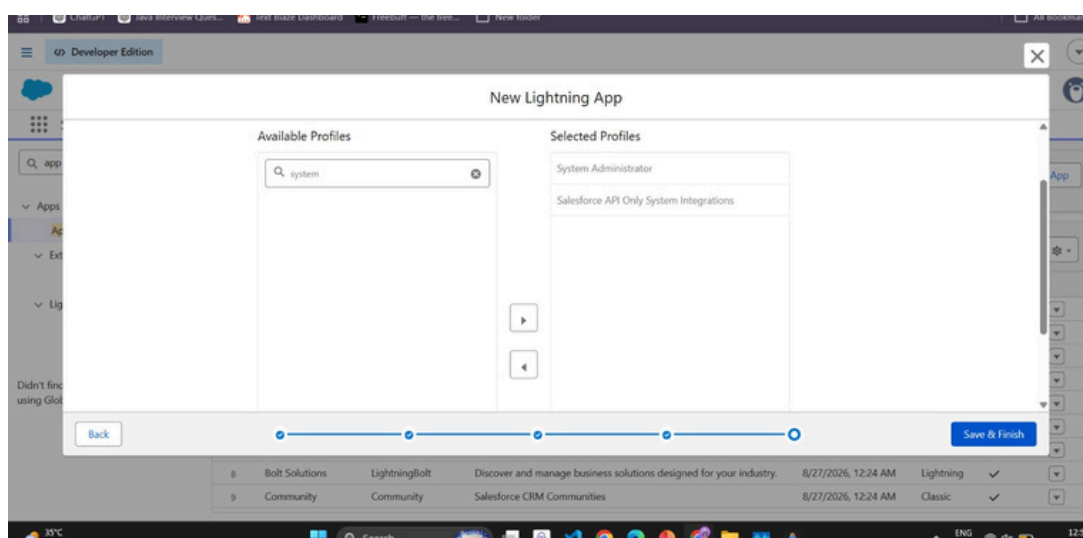
5.1 Lightning App Setup via App Manager

The WhatNext Vision Motors Lightning App acts as the main workspace for the project. It provides access to Vehicles, Dealers, Customers, Orders, Test Drives, and Service Requests from one application. This structure keeps the different parts of the dealership process connected while still giving each record type its own place.



The screenshot displays the Salesforce Lightning Experience App Manager interface. The left sidebar shows the navigation menu with 'Setup' and 'Home' tabs. The main content area is titled 'Lightning Experience App Manager' and shows a list of 31 apps. The list is sorted by App Name and filtered by All app menu items - TabSet Type, App Type. The table below represents the data shown in the screenshot:

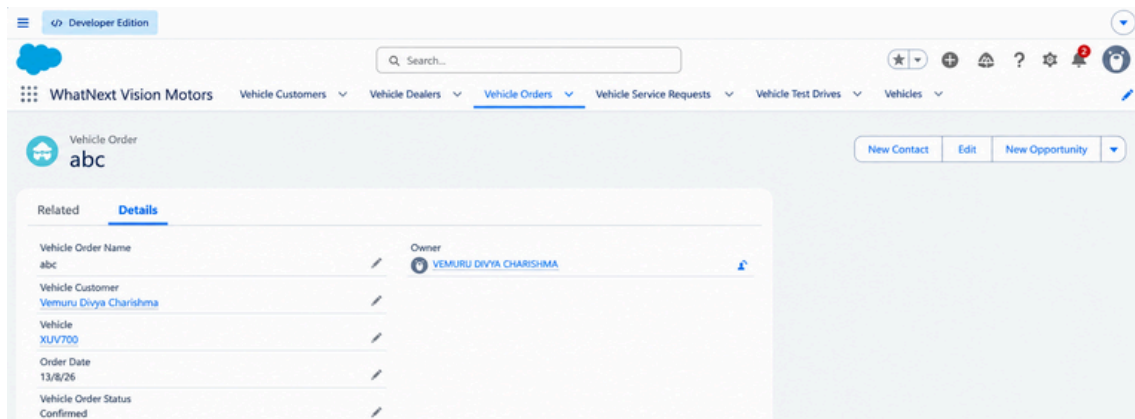
App Name	Developer Name	Description	Last Modified Date	App Type	Visible
1 Agentforce Grid	AIWorkbench	Access Agentforce Grid	8/27/2026, 12:34 AM	Lightning	✓
2 Agentforce Studio	AgentforceStudio	Agentforce Studio	8/27/2026, 12:34 AM	Lightning	✓
3 All Tabs	AllTabSet		8/27/2026, 12:24 AM	Classic	✓
4 Analytics Studio	Insights	Build CRM Analytics dashboards and apps	8/27/2026, 12:24 AM	Classic	✓
5 App Launcher	AppLauncher	App Launcher tabs	8/27/2026, 12:24 AM	Classic	✓
6 Approvals	Approvals	Manage approvals and approval flows	8/27/2026, 12:24 AM	Lightning	✓
7 Automation	FlowsApp	Automate business processes and repetitive tasks.	8/27/2026, 12:33 AM	Lightning	✓
8 Bolt Solutions	LightningBolt	Discover and manage business solutions designed for your industry.	8/27/2026, 12:24 AM	Lightning	✓
9 Community	Community	Salesforce CRM Communities	8/27/2026, 12:24 AM	Classic	✓



The screenshot displays the 'New Lightning App' setup window. The window is titled 'New Lightning App' and shows a list of available profiles and selected profiles. The 'Available Profiles' list includes 'System Administrator' and 'Salesforce API Only System Integrations'. The 'Selected Profiles' list includes 'System Administrator' and 'Salesforce API Only System Integrations'. The window also features a 'Back' button and a 'Save & Finish' button.

5.2 Page Layouts & Dynamic Forms

Page layouts were set up to organize the fields and related information on each record so that users can easily view the details they need. Dynamic Forms were used for fields that need to appear only under certain conditions. This means users see the relevant fields based on the information or values in the record, instead of seeing every field at the same time.



5.3 Management of USER

Users and records were set up based on the application's access requirements, and the record-entry forms were tested with sample data to ensure they were easy to use.

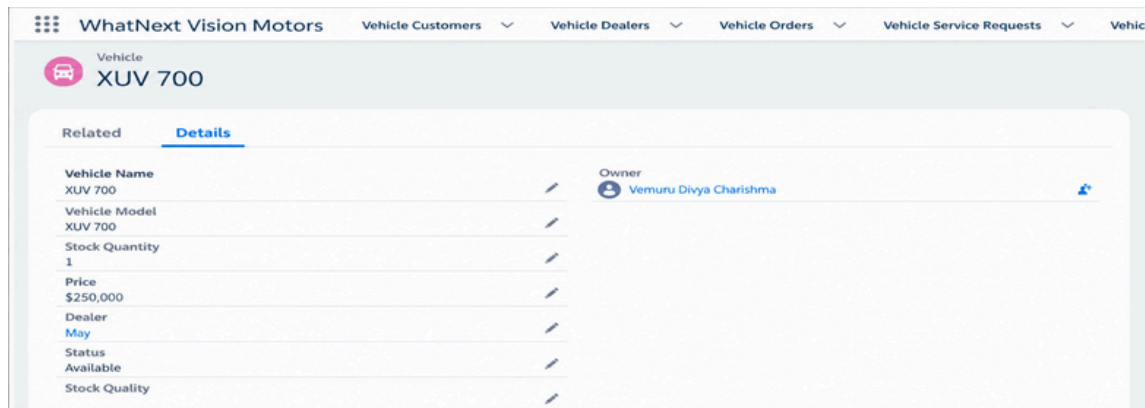
A screenshot of a Salesforce form titled 'Information'. The form contains the following fields: 'Vehicle Customer Name' (Vemuru Divya Charishma), 'Vehicle Customer Email' (divya.charishma@rmkec.ac.in), 'Phone' (empty), 'Preferred Vehicle Type' (XUV700), and 'Vehicle Customer Address' (empty). Each field has a blue refresh icon to its right.

5.4 Reports and Dashboards

Reports were created to organize and display Salesforce records in a structured way, such as vehicle stock levels, order status, and test-drive details. Dashboards present this information using visual charts and summaries, making it easier for users to understand business activities and monitor important data quickly.

5.5 Lightning Record Pages

Lightning Record Pages were configured to display important record information in one place. They combine record details, the Highlights Panel, and Related Lists, allowing users to view and access relevant information quickly without navigating through multiple pages.



6. Phase 4 — Data Migration, Testing & Security

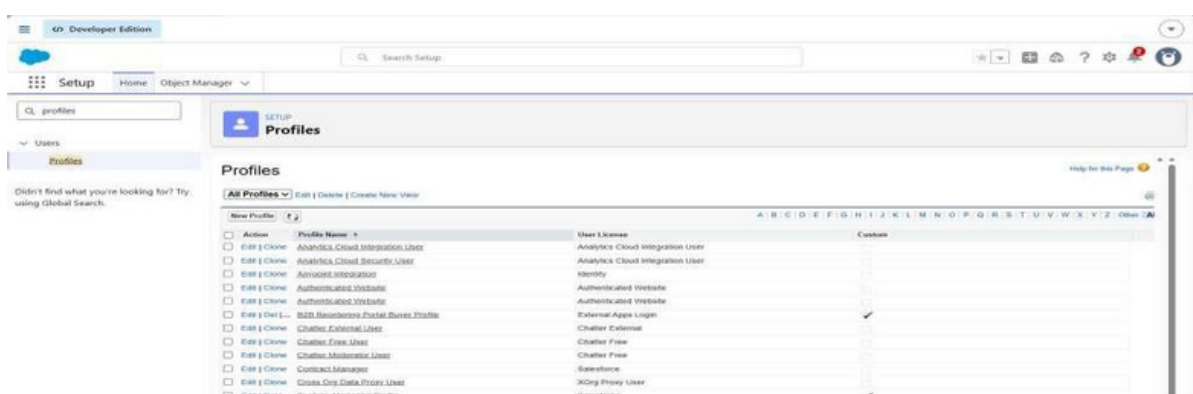
The fourth phase focused on adding sample data, checking that the application behaved as expected, and configuring Salesforce security. Testing was performed around record creation, stock validation, automated reminders, and test-drive status changes.

6.1 Data Loading

Sample data was prepared before being entered into Salesforce. The process involved preparing records, matching the data with Salesforce fields, loading the records, checking the saved values, and verifying the relationships between connected records. Depending on the object and amount of data, Salesforce Data Import Wizard or Data Loader can be used.

6.2 Profiles

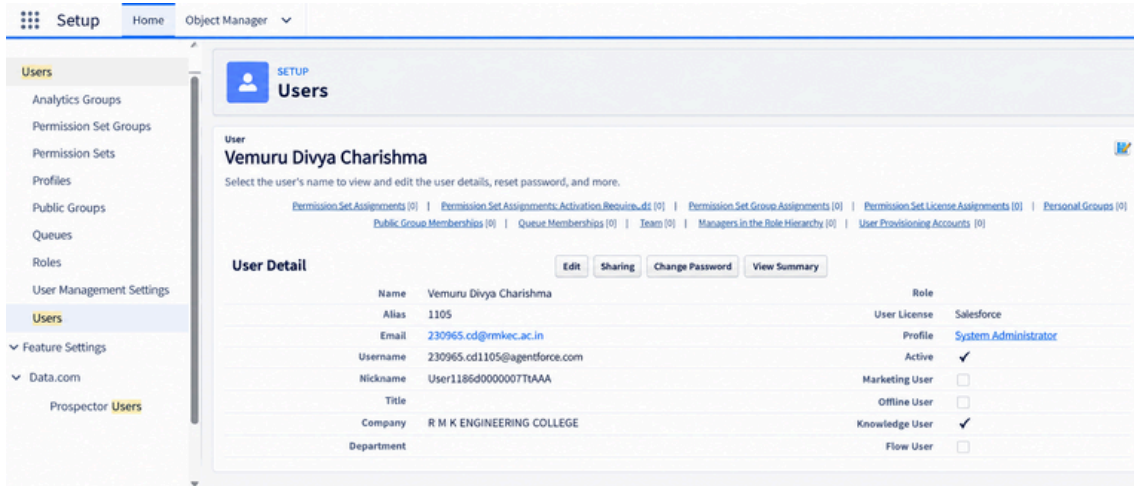
Profiles provide the basic level of access for Salesforce users. They control access to applications, objects, fields, and other Salesforce functionality.



6.3 Roles and Role Hierarchy

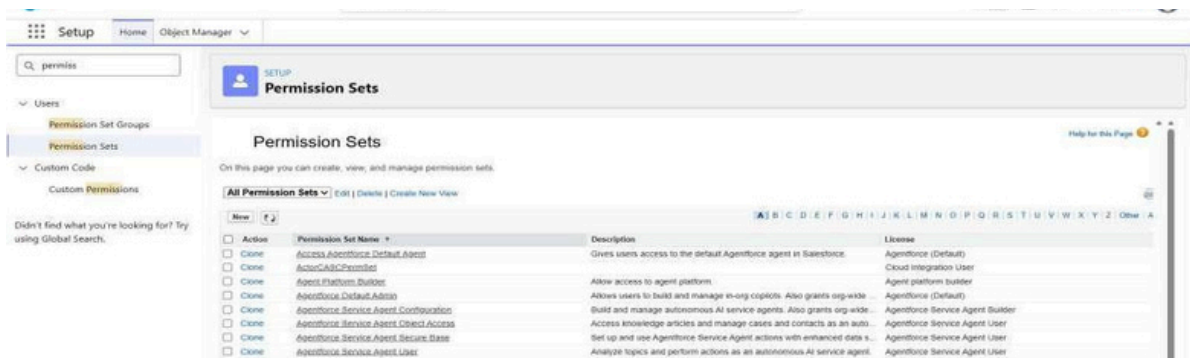
The role hierarchy represents the organizational structure used for record-access considerations. A representative structure used for the project is:

CEO → Sales Manager → Sales Representative



6.4 Permission Sets

Permission Sets were used to provide users with additional permissions and access to specific Salesforce features beyond the permissions available through their assigned profiles.



6.5 Sharing rules

Sharing configuration controls how records are accessed by different users. The sharing settings were configured according to user roles, ensuring that users can access the information they need while preventing unnecessary access to restricted records.

6.6 TestCases

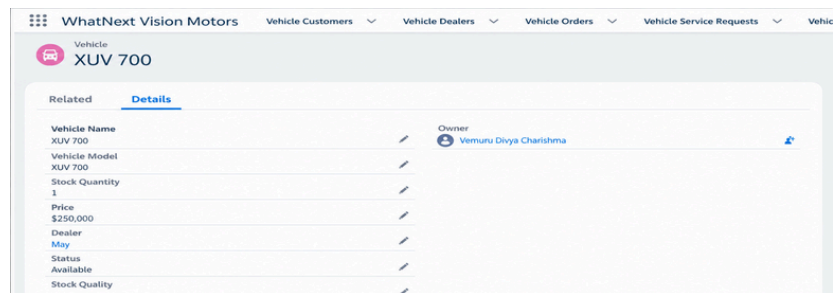
Test cases were prepared for the core Salesforce features — record creation, validation, automated flows, and triggers. Each test case was verified using appropriate input data and corresponding output evidence.

Test Case 1 — Creation of a Vehicle

Objective: Verify that a valid vehicle record can be created successfully.

Test Data:

- Vehicle Name: XUV700
- Model: 700
- Stock Quantity: 1
- Price: ₹2,50,000
- Status: Available
- Dealer: May

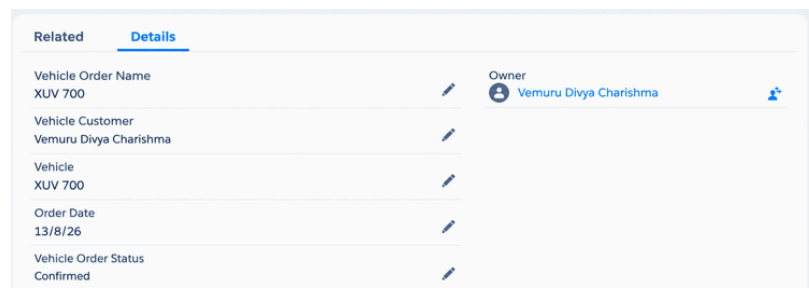


Test Case 2 — Stock Validation on Order Confirmation

Objective: Verify that the stock validation rule correctly controls order confirmation, allowing an order to be marked as Confirmed only when the associated vehicle has sufficient available stock.

Test Data:

- Vehicle Customer: Vemuru Divya Charishma
- Vehicle: XUV700
- Order Date: 13/8/26
- Vehicle Stock Quantity: 1
- Order Status: Confirmed

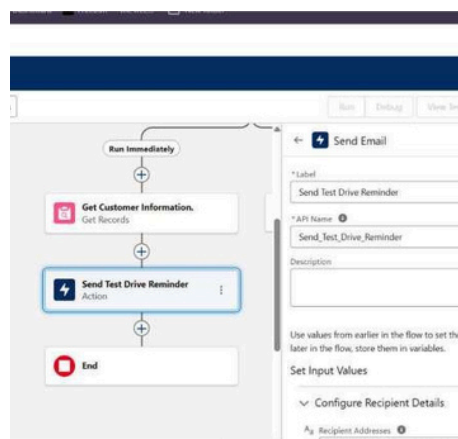


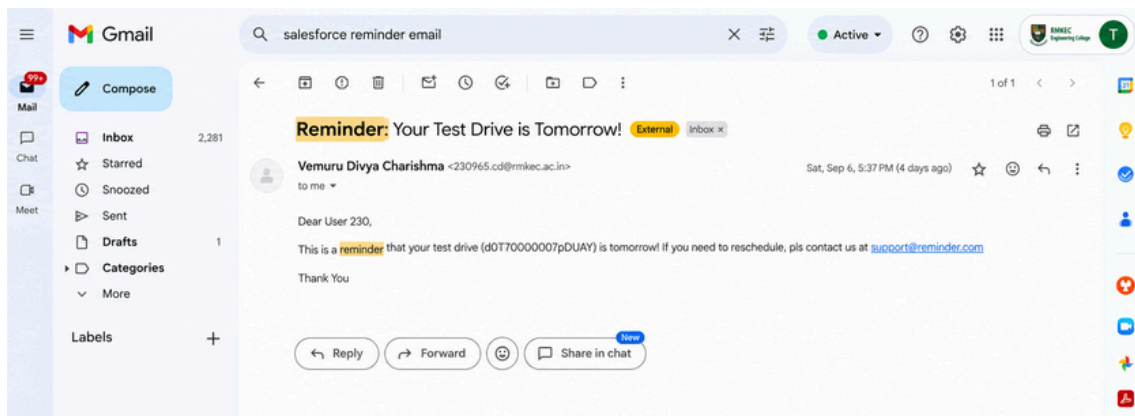
Test Case 3 — Test Drive Reminder Email

Objective: Verify the automated reminder process for an upcoming test drive.

Test Data:

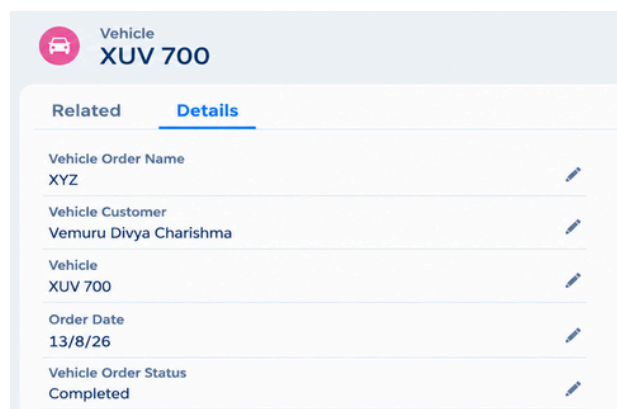
- Customer: Existing customer
- Customer Email: Valid email
- Status: Scheduled
- Test Drive Date: Following day





Test Case 4 — Test Drive Completion

Objective: Verify that a test drive record can progress through its lifecycle and be marked Completed.



7. Phase 5 — Deployment, Documentation & Maintenance

The final phase covers how the Salesforce configuration can be moved to the required environment, monitored after deployment, and maintained when issues occur.

7.1 Deployment Strategy

Salesforce components can be moved between environments using supported methods such as Change Sets, packages, or CI/CD tools. For this project, the deployment scope includes:

- Custom objects
- Custom fields
- Relationships
- Validation rules
- Flows
- Apex classes
- Apex triggers
- Lightning pages
- Reports and dashboards

7.2 Maintenance & Monitoring

Salesforce monitoring tools can be used to identify problems in the application. Important areas include Apex Jobs for monitoring batch or scheduled processing, Debug Logs for investigating unexpected Apex behaviour, Flow Errors for identifying failed automation, and user-access reviews for checking permissions periodically.

7.3 Troubleshooting Approach

When an issue occurs, the problem is first identified and reproduced with controlled data. The related object, fields, validation rules, Flow, permissions, or Apex logic are then reviewed. Logs are checked where required, the cause is corrected, and the original scenario is tested again. After the fix works as expected, the corrected configuration can be deployed to the required environment.

8. Conclusion

The WhatNext Vision Motors project brings the main activities of an automotive dealership into one Salesforce CRM application. The project connects vehicles, customers, dealers, orders, test drives, and service requests so that related information can be managed together instead of being handled separately. Stock validation helps prevent incorrect order confirmation, Flow automates dealer assignment and test-drive reminders, and Apex supports the order-processing logic that requires programmatic control. Lightning pages, reports, dashboards, and Salesforce security features make the system more organized and easier to manage. Overall, the project provided practical experience in combining Salesforce configuration features such as Objects, Fields, Relationships, Validation Rules, Flow, Reports, and Dashboards with Apex classes and triggers to build a CRM solution around a real automotive dealership workflow.